

Hybrid Quantum Neural Network (HQNN) 使用手册

李绍直

邮箱: lishaozhiphys@gmail.com

1. 主要功能和技术特点

HQNN 是一款混合式量子-经典神经网络计算软件, 旨在于经典计算机上高效模拟量子人工智能算法。用户可以在输入文件 INCARZ 中设置任意量子—经典神经网络结构和训练样本信息, 利用 HQNN 进行机器学习任务。该软件的机器学习部分程序是封闭式的, 而量子线路函数是开源式的。因此, 用户可以根据需要设计不同的量子线路, 研究不同量子线路对机器学习性能的影响。本软件主要面向高校、科研院所从事量子算法研究的人员, 以及量子计算公司的算法工程师。

当前, 量子人工智能领域广泛采用加拿大 Xanadu 公司开发的 PennyLane AI 软件进行计算, 但其并行计算能力有限, 导致大规模模拟耗时较长。为解决这一问题, HQNN 采用 OpenMP 技术, 在数据层面实现并行处理, 显著提升了计算效率。

图 1 展示了 HQNN 与 PennyLane AI 在相同任务下的性能对比 (基于 MNIST 数据集模拟混合量子-经典卷积神经网络):

- **红色柱状图:** PennyLane AI (单核) 计算一个 epoch 所需时间
- **蓝色柱状图:** HQNN (单核) 计算一个 epoch 所需时间

结果显示, HQNN 在单核性能上比 PennyLane AI 快约十倍。此外, HQNN 支持多核并行, 计算时间随 CPU 数量增加线性减少。

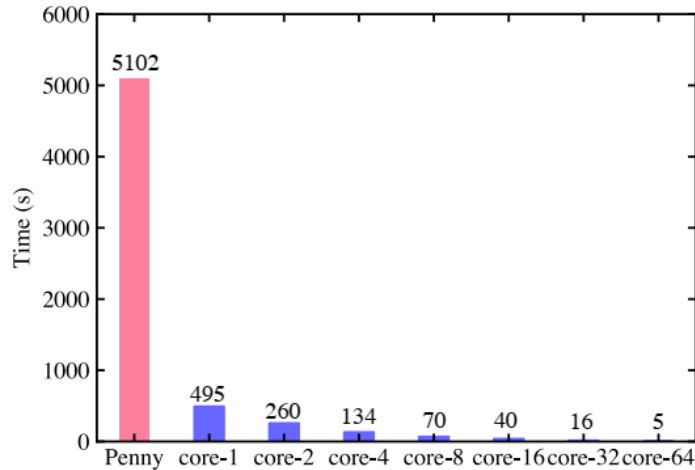


图 1 HQNN 与 PennyLane AI 软件的性能对比。

2. 硬件环境

- 处理器: AMD Ryzen 9 9950X 16 核
- 内存: 64 GB
- 安装所需空间: 429 MB
- 运行所需空间: 由训练数据大小决定
- 显卡: 集成显卡即可

3. 软件环境

- 操作系统：Ubuntu 22.0.4
- 需安装：
 - g++ 10.0 或更高版本
 - HDF5, 1.10.6 或更高版本
 - Boost, 1.79.0 或更高版本,
 - Eigen3, 3.4.0 或更高版本,
 - Qulacs, 量子模拟器, 下载地址 <https://github.com/qulacs/qulacs>

4. 编程语言

该软件是基于 C++语言编写，采用 OpenMP 实行并行计算。软件通过 makefile 进行编译和安装。

5. 源程序量

该软件现共有 3691 行 C++代码。makefile 安装代码 38 行。

6. 软件内容

HQNN 目录结构如图 2 所示，其中包括六个文件夹和三个文件。

- /dataset: 存有经典机器学习任务的训练样本（MNIST）。
- /include 与/src: 存放 C++源文件。
- /obj: 编译后生成的目标文件存放位置，如图 3 所示。

名称	修改日期	类型	大小
build	2026/1/5 21:49	文件夹	
dataset	2026/1/5 21:50	文件夹	
include	2026/1/5 21:50	文件夹	
obj	2026/1/5 21:57	文件夹	
src	2026/1/5 21:55	文件夹	
test	2026/1/5 21:49	文件夹	
makefile_step2	2026/1/5 21:49	文件	2 KB
quantum_circuit	2026/1/5 21:49	C++ 源文件	3 KB

图 2 软件内部文件

名称	修改日期	类型	大小
 backward.o	2026/1/5 21:56	O 文件	1,914 KB
 buildmodel.o	2026/1/5 21:55	O 文件	2,565 KB
 define_parameters.o	2026/1/5 21:55	O 文件	1,264 KB
 forward.o	2026/1/5 21:56	O 文件	1,798 KB
 hybrid.o	2026/1/5 21:56	O 文件	2,096 KB
 io.o	2026/1/5 21:57	O 文件	2,389 KB
 layer.o	2026/1/5 21:57	O 文件	8,281 KB
 local_hdf5.o	2026/1/5 21:56	O 文件	589 KB
 nnmodel.o	2026/1/5 21:56	O 文件	4,811 KB
 operation.o	2026/1/5 21:57	O 文件	652 KB
 optimizer.o	2026/1/5 21:57	O 文件	1,876 KB
 perform_test.o	2026/1/5 21:56	O 文件	2,711 KB
 read_checkpoint.o	2026/1/5 21:55	O 文件	891 KB
 read_sample.o	2026/1/5 21:55	O 文件	1,551 KB

图 3 /obj 文件夹内部文件

```
➤ /build: 存放 makefile_step1 用于编译。具体内容如下所示
=====
HDF5_include=/usr/include/hdf5/serial
HDF5_library=/usr/lib/x86_64-linux-gnu/hdf5/serial -lhdf5_serial -lhdf5_serial_cpp

qulacs_home=/home/szl/Documents/library/qulacs
qulacs_include=$(qulacs_home)/include
qulacs_library=$(qulacs_home)/lib -lvqcsim_static -lcppsim_static -lcsim_static

BOOST_INCLUDE=/usr/include/boost
BOOST_LIBRARY=/usr/lib/x86_64-linux-gnu -lboost_program_options

EIGEN3_INCLUDE=/usr/include/eigen3

CXXFLAGS = -w -g -std=c++17 -O3 -I$(HDF5_include) -I$(BOOST_INCLUDE) -
I$(EIGEN3_INCLUDE) -I$(qulacs_include)
CFLAGS = -L$(qulacs_library) -L$(HDF5_library) -L$(BOOST_LIBRARY) -fopenmp

# Find all c++ source files
OBJDIR=../obj
SRCDIR=../src
CPP_SOURCES := $(wildcard $(SRCDIR)/*.cpp)

# convert the *.cpp filenames to *.o
CPP_OBJECTS := $(CPP_SOURCES:$(SRCDIR)/%.cpp=$(OBJDIR)/%.o)
```

all: \$(CPP_OBJECTS)

\$(OBJDIR)/%.o: \$(SRCDIR)/%.cpp
g++ \$(CXXFLAGS) -c \$< -o \$@ \$(CFLAGS)

clean:
rm \$(OBJDIR)/*.o

=====

在 makefile 中， 用户需要给定 HDF5, qulacs, Boost, Eigen3 的路径。修改路径之后， 用户需要通过 terminal 命令编译文件， 即

makefile -f makefile_step1

- /test: 包含示例程序及配置文件， 如图 4 所示。用户可以通过 terminal 运行这个软件， 即

./HQNN INCAR

名称	修改日期	类型	大小
case1	2026/1/6 11:32	文件夹	
case2	2026/1/6 11:32	文件夹	

图 4 /test 内文件

7. 软件安装方法

HQNN 需要分成两步安装。

第一步：编译核心库

- 在/build/makefile_step1 中设置 HDF5, Boost, Eigen3, 和 Qulacs 路径
- 在/build 文件夹下使用 makefile 编译软件， 产生的可执行文件保存在 /obj 中

第二步：配置并编译用户量子线路

- 拷贝根目录下 quantum_circuit.cpp （如图 2 所示）到工作文件夹/work
- 用户需要根据自己的需求修改 quantum_circuit.cpp 中量子线路。此处量子线路使用 Qulacs 计算软件
- 拷贝根目录下 makefile_step2 （如图 2 所示）到工作文件夹/work, 修改内部 HDF5, Boost, Eigen3, Qulacs 路径以及 OBJDIR 路径
- 在工作文件夹下用 makefile 编译文件， 产生 **HQNN** 可执行文件。

第二步中的 makefil_step2 如下所示， 其中红色文字部分需要修改。

=====

HDF5=/usr/include/hdf5/serial
HDF5_include=/usr/include/hdf5/serial
HDF5_library=/usr/lib/x86_64-linux-gnu -lhdf5_serial -lhdf5_serial_cpp

qulacs_home=/home/tjzs/Documents/library/qulacs

```

qulacs_include=$(qulacs_home)/include
qulacs_library=$(qulacs_home)/lib -lvqcsim_static -lcppsim_static -lcsim_static

BOOST_HOME=/home/tjzs/Documents/library/local
BOOST_INCLUDE=$(BOOST_HOME)/include/boost
BOOST_LIBRARY=$(BOOST_HOME)/lib -lboost_program_options

EIGEN3_HOME=/home/tjzs/Documents/library/local/include
EIGEN3_INCLUDE=$(EIGEN3_HOME)/eigen3

CXXFLAGS = -w -g -std=c++17 -O3 -I$(HDF5_include) -I$(BOOST_INCLUDE) -
I$(EIGEN3_INCLUDE) -I$(qulacs_include)
CFLAGS = -L$(qulacs_library) -L$(HDF5_library) -L$(BOOST_LIBRARY) -fopenmp

OBJDIR=/home/tjzs/Documents/library/HQCNN/obj

LOCAL_SRC=quantum_circuit.cpp
LOCAL_OBJ=quantum_circuit.o

OBS = $(wildcard $(OBJDIR)/*.o)

all: HQNN

HQNN: $(LOCAL_OBJ) $(OBS)
    g++ $(CXXFLAGS) -o $@ $^ $(CFLAGS)

$(LOCAL_OBJ): $(LOCAL_SRC)
    g++ $(CXXFLAGS) -c $< -o $@ $(CFLAGS)

clean:
    rm quantum_circuit.o HQNN
=====

```

8. 软件使用方法

HQNN 采用 OPENMP 实现并行。因此，在执行程序之前，需要设置 thread 数量。打开 terminal，用如下命令设置 thread 数量为 4，

```
export OMP_NUM_THREADS=4
```

在工作目录执行

```
./HQNN INCAR
```

其中，INCAR 为配置文件，包含机器学习样本信息以及神经网络结构参数。/test 中存放两个案例。用户可以基于这两个案例学习 HQNN 的使用方法。

8.1 案例一

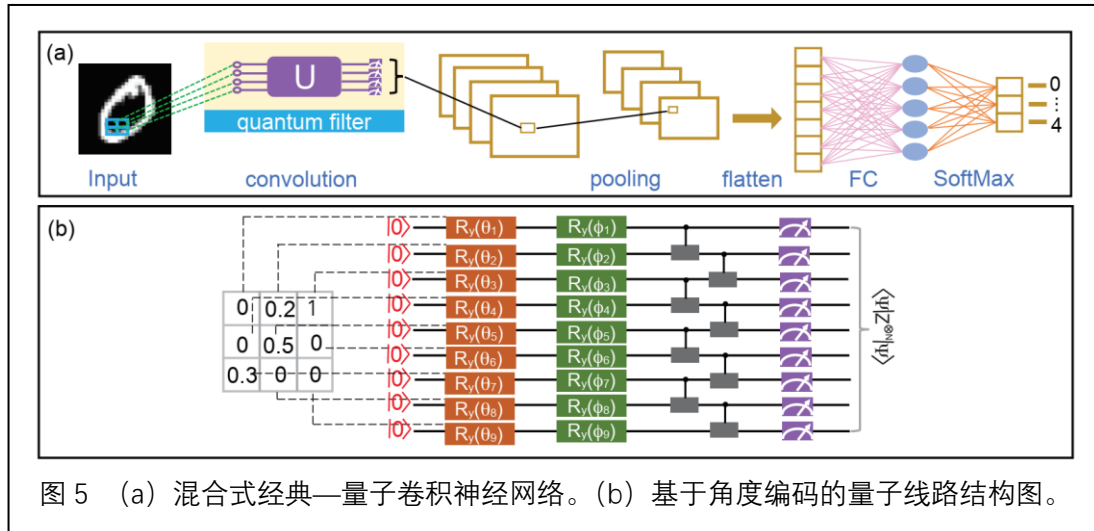


图5 (a) 混合式经典—量子卷积神经网络。(b) 基于角度编码的量子线路结构图。

/test/case1 中存放着经典—量子卷积神经网络的使用案例。这个例子使用混合式经典—量子卷积神经网络对 MNIST（手写数字 0-4）数据集进行判别。这个神经网络包含 5 层网络（如图 5 (a)所示）。其中第一层为量子卷积网络，第二层为 Pooling 层，第三层为展开层，第四层和第五层是全链接网络。第五层即输出层，含有 5 个神经元。用户可以自行调节层数以及每层对应的类型和神经元数量。在量子卷积网络层，用户可以根据需要定义量子线路数量 (cnn1.NQC) 以及量子线路上的变量数量 (cnn1.Qwsize)。具体的量子线路需要在 quantum_circuit.cpp 文件里 `double quantum_filter(int NQC, Eigen::Tensor<double,3> &input, Eigen::Tensor<double,1> &weight) { ... }` 函数中进行设置。因此，quantum_filter 是一个开源函数，用户可以根据自己的需求设计不同的量子线路结构。

Case1 中的 INCAR 具体内容如下所示。

```
=====
# all letters should be lowercase
# input training data path
train_file = ../dataset/mnist_train.dat
# input testing data path
test_file = ../dataset/mnist_test.dat
# define number of neural network layers
NNlayer = 5
# define the type of each layer, flatten, dense, pool, or cnn
layer1 = cnn
layer2 = pool
layer3 = flatten
layer4 = dense
layer5 = dense

# define jobid for labeling
jobid = 1
# define the size of training data set
Ntrain = 200
# define the width of input image
```

```
image.col = 28
# define the length of input image
image.row = 28
# define the depth of input image
image.depth = 1
# define epoch number
epochs = 70
# define the size of testing data set
Ntest = 64
# define batch size
batch = 64

# define use classical or quantum algorithm
cnn1.device = quantum
# define kernel size in CNN
cnn1.kernelx = 3
cnn1.kernely = 3
# define stride size in CNN
cnn1.stridex = 1
cnn1.stridey = 1
# define pad size in CNN
cnn1.pad = 0
# define feature size in first layer
cnn1.feature = 16
# define activation function. If use quantum, set it as none
cnn1.activation = none
# define the number of quantum circuit. If use classical algorithm, the next two variables will
not be used.
cnn1.NQC = 9
# define the number of variables in a quantum circuit
cnn1.Qwsize = 9

# define pool layer parameters
pool1.kernelx = 2
pool1.kernely = 2
pool1.stridex = 2
pool1.stridey = 2
pool1.pad = 0
pool1.activation = mean

# define feature size in the third layer
dense1.feature = 64
dense1.activation = tanh
```

```
# define feature size in the fourth layer
```

```
dense2.feature = 5
```

```
dense2.activation = linear
```

```
# define the output size
```

```
nn.out = 5
```

```
# define the loss function type, support categoricalcrossentropy and MSE
```

```
loss.type = categoricalcrossentropy
```

```
# do not save hidden layer information for training and testing datasets
```

```
keep.train = no
```

```
keep.test = no
```

```
=====
```

/test/case1/quantum_circuit.cpp 给出了图 5（b）中展示的量子线路的代码。此处经典数据是通过角度编码转入量子线路的。具体的程序如下所示。

```
=====
```

```
// File Name "quantum_circuit.cpp"
```

```
#include<iostream>
```

```
#include<fstream>
```

```
#include<sstream>
```

```
#include<random>
```

```
#include<vector>
```

```
#include<complex>
```

```
#include<boost/program_options.hpp>
```

```
#include <unsupported/Eigen/CXX11/Tensor>
```

```
#include<chrono>
```

```
#include<cppsim/state.hpp>
```

```
#include<cppsim/circuit.hpp>
```

```
#include<cppsim/observable.hpp>
```

```
#include <vqcsim/parametric_circuit.hpp>
```

```
#include <cppsim/utility.hpp>
```

```
#include <cppsim/circuit_optimizer.hpp>
```

```
#include <cppsim/gate_factory.hpp>
```

```
#include <cppsim/gate_merge.hpp>
```

```
#include <cppsim/gate_matrix.hpp>
```

```
using namespace std;
```

```
double quantum_filter(int NQC, Eigen::Tensor<double,3> &input,Eigen::Tensor<double,1>
&weight){
```

```
    int inputsize=input.dimension(0)*input.dimension(1)*input.dimension(2);
```

```
    int Lx = input.dimension(0);
```



```
int Ly = input.dimension(1);
int Lz = input.dimension(2);
std::vector<double> angle1(NQC), angle2(NQC);
for(int ix=0; ix<Lx; ++ix){
    for(int iy=0; iy<Ly; ++iy){
        int i=ix*3+iy;
        // angle encoding
        angle1[i]=input(ix,iy,0);
    }
}
for(int i=0; i<NQC; ++i){
    angle2[i]=weight(i);
}

// create an initial quantum state
QuantumState state(NQC);
state.set_zero_state();

// create a quantum circuit with NQC qubits
ParametricQuantumCircuit* circuit1 = new ParametricQuantumCircuit(NQC);
// apply rotation gates
for(int i=0; i<NQC; ++i){
    circuit1->add_parametric_RY_gate(i,M_PI*(angle1[i]+angle2[i]));
}
// apply CNOT gates
for(int i=0; i<NQC; i=i+2){
    int j=i+1;
    if(j>=NQC) j=j-NQC;
    circuit1->add_CNOT_gate(i,j);
}
for(int i=1; i<NQC; i=i+2){
    int j=i+1;
    if(j>=NQC) j=j-NQC;
    circuit1->add_CNOT_gate(i,j);
}
QuantumCircuitOptimizer optimizer;
optimizer.optimize_light(circuit1);

circuit1->update_quantum_state(&state);
// perform measurements
Observable observable(NQC);
std::string measurement="Z 0 Z 1 Z 2 Z 3 Z 4 Z 5 Z 6 Z 7 Z 8";
observable.add_operator(1.0,measurement);
std::complex<double> mean;
```

```

        mean=observable.get_expectation_value(&state);
        delete circuit1;

        return std::real(mean);
    }

```

关于量子线路的设置，用户需要熟悉 qulacs 软件的使用方法。具体用法请查询官网（<https://docs.qulacs.org/en/latest/>）。

运行程序 (./HQNN INCAR) 后，terminal 会显示如下画面。

```

=====
Hybrid Quantum Neural Network
code was developed by Shaozhi Li, 2026-01-01
A permission is required to use this code
Contact: lishaozhiphys@gmail.com
=====
Maximum number of threads: 32
read in parameters
NNlayer: 5(int)
Ntest: 64(int)
Ntrain: 200(int)
batch: 64(int)
cnn1.NQC: 9(int)
cnn1.Qwsize: 9(int)
cnn1.activation: none(string)
cnn1.device: quantum(string)
cnn1.encode: angle(string)
cnn1.feature: 16(int)
cnn1.kernelx: 3(int)
cnn1.kernely: 3(int)
cnn1.pad: 0(int)
cnn1.stridex: 1(int)
cnn1.stridey: 1(int)
dense1.activation: tanh(string)
dense1.feature: 64(int)
dense2.activation: linear(string)
dense2.feature: 5(int)
epochs: 1(int)
image.col: 28(int)
image.depth: 1(int)
image.row: 28(int)
jobid: 1(int)
keep.test: no(string)
keep.train: no(string)
layer1: cnn(string)
layer2: pool(string)
layer3: flatten(string)
layer4: dense(string)
layer5: dense(string)
loss.type: categoricalcrossentropy(string)
nn.out: 5(int)
pool1.activation: mean(string)
pool1.kernelx: 2(int)
pool1.kernely: 2(int)
pool1.pad: 0(int)
pool1.stridex: 2(int)
pool1.stridey: 2(int)
test_file: ../../dataset/mnist_test.dat(string)
train_file: ../../dataset/mnist_train.dat(string)
sucess read training dataset
success in read test dataset
finish reading

```

```
cnn
pool
flatten
dense
dense
build_model
=====
epoch = 1
epoch = 1 i= 0 loss = 1.60943
epoch = 1 i= 2 loss = 1.44874
epoch = 1 train loss = 1.65459 test loss = 1.20013
train accuracy = 0.244792 test accuracy = 0.765625
cost time = 171.126 seconds.
```

计算结束后，结果保存在 model_1.h5 文件里，model_1.h5 的数据结构如下所示。

```
=====
HDF5 "model_1.h5" {
FILE_CONTENTS {
group      /
group      /model
group      /model/cnn0
dataset    /model/cnn0/weight           #训练后的权重系数
group      /model/dense0
dataset    /model/dense0/biase
dataset    /model/dense0/weight
group      /model/dense1
dataset    /model/dense1/biase
dataset    /model/dense1/weight
group      /model/histogram
dataset    /model/histogram/historgam
group      /model/parameter           #保存所有输入参数
dataset    /model/parameter/NNlayer
dataset    /model/parameter/Ntest
dataset    /model/parameter/Ntrain
dataset    /model/parameter/batch
dataset    /model/parameter/cnn1.NQC
dataset    /model/parameter/cnn1.Qwsize
dataset    /model/parameter/cnn1.activation
dataset    /model/parameter/cnn1.device
dataset    /model/parameter/cnn1.encode
dataset    /model/parameter/cnn1.feature
dataset    /model/parameter/cnn1.kernelx
dataset    /model/parameter/cnn1.kernely
dataset    /model/parameter/cnn1.pad
dataset    /model/parameter/cnn1.stridex
dataset    /model/parameter/cnn1.stridey
dataset    /model/parameter/dense1.activation
dataset    /model/parameter/dense1.feature
dataset    /model/parameter/dense2.activation
dataset    /model/parameter/dense2.feature
dataset    /model/parameter/epochs
```

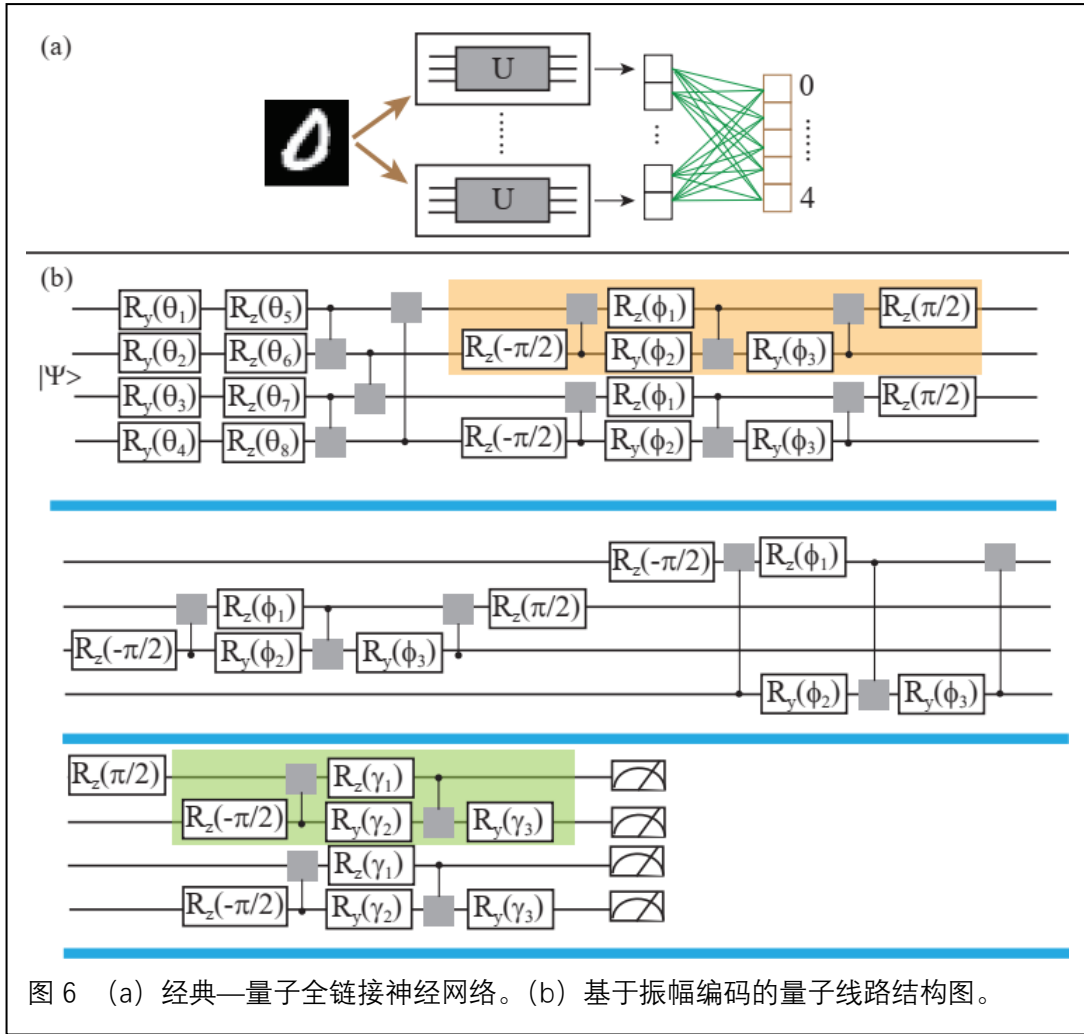
```
dataset    /model/parameter/image.col
dataset    /model/parameter/image.depth
dataset    /model/parameter/image.row
dataset    /model/parameter/jobid
dataset    /model/parameter/keep.test
dataset    /model/parameter/keep.test.sample.num
dataset    /model/parameter/keep.train
dataset    /model/parameter/keep.train.sample.num
dataset    /model/parameter/layer1
dataset    /model/parameter/layer2
dataset    /model/parameter/layer3
dataset    /model/parameter/layer4
dataset    /model/parameter/layer5
dataset    /model/parameter/loss.type
dataset    /model/parameter/nn.out
dataset    /model/parameter/pool1.activation
dataset    /model/parameter/pool1.kernelx
dataset    /model/parameter/pool1.kernely
dataset    /model/parameter/pool1.pad
dataset    /model/parameter/pool1.stridex
dataset    /model/parameter/pool1.stridey
dataset    /model/parameter/test_file
dataset    /model/parameter/train_file
group      /prediction
dataset    /prediction/test                #测试集预测结果和真实结果
dataset    /prediction/test_loss          #测试损失函数记录
dataset    /prediction/train              #训练集预测结果和真实结果
dataset    /prediction/train_loss         #训练损失函数记录
}
}
```

=====

/model/parameter/保存着所有 input 模型参数。/model/cnn0/weight, /model/dense0/weight 和 /model/dense1/weight 中保存着训练好的模型权重参数。/prediction/train 和 /prediction/test 中保存着训练和测试数据集的预测值和真实值。/prediction/train_loss 和 /prediction/test_loss 保存着每个 epoch 中训练样本和测试样本的损失函数值。用户可通过 HDF5 查看工具或者相应编程接口读取和分析结果。

8.2 案例 2

/test/case2 中存放着经典—量子全链接神经网络，如图 6 所示。案例 2 的 INCAR 如下所示。



```
=====
```

```
# all letters should be lowercase
```

```
# input training data path
```

```
train_file = ../../dataset/mnist_train.dat
```

```
# input testing data path
```

```
test_file = ../../dataset/mnist_test.dat
```

```
# define number of neural network layers
```

```
NNlayer = 4
```

```
# define the type of each layer, flatten, dense, pool, or cnn
```

```
layer1 = cnn
```

```
layer2 = flatten
```

```
layer3 = dense
```

```
layer4 = dense
```

```
# define jobid for labeling
```

```
jobid = 1
```

```
# define the size of training data set
```

```
Ntrain = 200
```

```
# define the width of input image
image.col = 28
# define the length of input image
image.row = 28
# define the depth of input image
image.depth = 1
# define epoch number
epochs = 1
# define the size of testing data set
Ntest = 64
# define batch size
batch = 64

# define use classical or quantum algorithm
cnn1.device = quantum
# define kernel size in CNN
cnn1.kernelx = 28
cnn1.kernely = 28
# define stride size in CNN
cnn1.stridex = 1
cnn1.stridey = 1
# define pad size in CNN
cnn1.pad = 0
# define feature size in first layer
cnn1.feature = 16
# define activation function. If use quantum, set it as none
cnn1.activation = none
# define the number of quantum circuit. If use classical algorithm, the next two variables will
not be used.
cnn1.NQC = 10
# define the number of variables in a quantum circuit
cnn1.Qwsiz = 26

# define feature size in the third layer
dense1.feature = 64
dense1.activation = tanh

# define feature size in the fourth layer
dense2.feature = 5
dense2.activation = linear

# define the output size
nn.out = 5
```

```
# define the loss function type
```

```
loss.type = categoricalcrossentropy
```

```
# do not save hidden layer information for training and testing datasets
```

```
keep.train = no
```

```
keep.test = no
```

```
=====
```

图 6(b) 中量子线路的实现代码存放于/test/case2/quantum_circuit.cpp。具体代码如下所示。

```
=====
```

```
#include<iostream>
```

```
#include<fstream>
```

```
#include<sstream>
```

```
#include<random>
```

```
#include<vector>
```

```
#include<complex>
```

```
#include<boost/program_options.hpp>
```

```
#include <unsupported/Eigen/CXX11/Tensor>
```

```
#include<chrono>
```

```
#include<cppsim/state.hpp>
```

```
#include<cppsim/circuit.hpp>
```

```
#include<cppsim/observable.hpp>
```

```
#include <vqcsim/parametric_circuit.hpp>
```

```
#include <cppsim/utility.hpp>
```

```
#include <cppsim/circuit_optimizer.hpp>
```

```
#include <cppsim/gate_factory.hpp>
```

```
#include <cppsim/gate_merge.hpp>
```

```
#include <cppsim/gate_matrix.hpp>
```

```
using namespace std;
```

```
// quantum fully-connected network circuit
```

```
void quantum_NN_circuit(const int NQC, ParametricQuantumCircuit* circuit,
std::vector<double> &angle){
```

```
    for(int i=0; i<NQC; ++i){
```

```
        circuit->add_parametric_RY_gate(i,angle[i]*M_PI);
```

```
    }
```

```
    for(int i=0; i<NQC; ++i){
```

```
        circuit->add_parametric_RZ_gate(i,angle[i+NQC]*M_PI);
```

```
    }
```

```

    for(int i=0; i<NQC; ++i){
        int indexi=i;
        int indexj=i+1;
        if(indexj>=NQC) indexj=indexj-NQC;

        circuit->add_CNOT_gate(indexi,indexj);
    }

    QuantumCircuitOptimizer optimizer;
    optimizer.optimize_light(circuit);

}

// quantum convolutional network circuit
void quantum_cov_circuit(const int shift, const int NQC, ParametricQuantumCircuit*
circuit,std::vector<double> &angle){
    for(int i=0; i<NQC; i=i+2){
        int indexi=i+shift;
        int j=i+1;
        if(j>=NQC) j=j-NQC;
        int indexj=j+shift;
        circuit->add_parametric_RZ_gate(indexj,-0.5*M_PI);
        circuit->add_CNOT_gate(indexj,indexi);
        circuit->add_parametric_RZ_gate(indexi,angle[0]*M_PI);
        circuit->add_parametric_RY_gate(indexj,angle[1]*M_PI);
        circuit->add_CNOT_gate(indexi,indexj);
        circuit->add_parametric_RY_gate(indexj,angle[2]*M_PI);
        circuit->add_CNOT_gate(indexj,indexi);
        circuit->add_parametric_RZ_gate(indexi,0.5*M_PI);
    }

    for(int i=1; i<NQC; i=i+2){
        int indexi=i+shift;
        int j=i+1;
        if(j>=NQC) j=j-NQC;
        int indexj=j+shift;
        circuit->add_parametric_RZ_gate(indexj,-0.5*M_PI);
        circuit->add_CNOT_gate(indexj,indexi);
        circuit->add_parametric_RZ_gate(indexi,angle[0]*M_PI);
        circuit->add_parametric_RY_gate(indexj,angle[1]*M_PI);
        circuit->add_CNOT_gate(indexi,indexj);
        circuit->add_parametric_RY_gate(indexj,angle[2]*M_PI);
        circuit->add_CNOT_gate(indexj,indexi);
        circuit->add_parametric_RZ_gate(indexi,0.5*M_PI);
    }
}

```



```

    QuantumCircuitOptimizer optimizer;
    optimizer.optimize_light(circuit);
}

// quantum pooling network circuit
void quantum_pool_circuit(const int shift, const int NQC, ParametricQuantumCircuit* circuit,
std::vector<double> &angle){
    for(int i=0; i<NQC; i=i+2){
        int indexi=i+shift;
        int j=i+1;
        int indexj=j+shift;
        if(indexj<NQC){
            circuit->add_parametric_RZ_gate(indexj,-0.5*M_PI);
            circuit->add_CNOT_gate(indexj,indexi);
            circuit->add_parametric_RZ_gate(indexi,angle[0]*M_PI);
            circuit->add_parametric_RY_gate(indexj,angle[1]*M_PI);
            circuit->add_CNOT_gate(indexi,indexj);
            circuit->add_parametric_RY_gate(indexj,angle[2]*M_PI);
        }
    }
    QuantumCircuitOptimizer optimizer;
    optimizer.optimize_light(circuit);
}

// quantum_filter function for the whole quantum circuit
double quantum_filter(int NQC, Eigen::Tensor<double,3> &input,Eigen::Tensor<double,1>
&weight){

    int inputsize=input.dimension(0)*input.dimension(1)*input.dimension(2);
    int Lx = input.dimension(0);
    int Ly = input.dimension(1);
    int Lz = input.dimension(2);
    std::vector<double> angle1(2*NQC), angle2(3), angle3(3);
    for(int i=0; i<2*NQC; ++i){
        angle1[i]=weight(i);
    }
    for(int i=0; i<3; ++i){
        angle2[i]=weight(i+2*NQC);
        angle3[i]=weight(i+2*NQC+3);
    }

    int hilbsize = std::pow(2,NQC);

```

```

std::vector<std::complex<double>> x(hilbsize,0.0);
int k=0;
double xnorm=0.0;
for(int ix=0; ix<Lx; ++ix){
    for(int iy=0; iy<Ly; ++iy){
        x[k]=sqrt(input(ix,iy,0));
        xnorm += input(ix,iy,0);
        k++;
    }
}

if(xnorm>0.0){
    xnorm=sqrt(xnorm);
    for(int i=0; i<Lx*Ly; ++i){
        x[i]=x[i]/xnorm;
    }

    QuantumState state(NQC);
    state.set_zero_state();
    state.load(x);                // Amplitude encoding

    ParametricQuantumCircuit* circuit1 = new ParametricQuantumCircuit(NQC);
    quantum_NN_circuit(NQC,circuit1,angle1);
    quantum_cov_circuit(0,NQC,circuit1,angle2);
    quantum_pool_circuit(0,NQC,circuit1,angle3);
    circuit1->update_quantum_state(&state);

    Observable observable(NQC);
    std::string measurement="I 0 Z 1 I 2 Z 3 I 4 Z 5 I 6 Z 7 I 8 Z 9";
    observable.add_operator(1.0,measurement);
    std::complex<double> mean;
    mean=observable.get_expectation_value(&state);
    delete circuit1;

    return std::real(mean*xnorm);
}else{
    return 0.0;
}
}

```

=====

运行软件（./HQNN INCAR）之后，在终端输出结果如下所示。

```
=====
Hybrid Quantum Neural Network
code was developed by Shaozhi Li, 2026-01-01
A permission is required to use this code
Contact: lishaozhiphys@gmail.com
=====
Maximum number of threads: 32
read in parameters
NNlayer: 4(int)
Ntest: 64(int)
Ntrain: 200(int)
batch: 64(int)
cnn1.NQC: 10(int)
cnn1.Qwsize: 26(int)
cnn1.activation: none(string)
cnn1.device: quantum(string)
cnn1.feature: 16(int)
cnn1.kernelx: 28(int)
cnn1.kernely: 28(int)
cnn1.pad: 0(int)
cnn1.stridex: 1(int)
cnn1.stridey: 1(int)
dense1.activation: tanh(string)
dense1.feature: 64(int)
dense2.activation: linear(string)
dense2.feature: 5(int)
epochs: 1(int)
image.col: 28(int)
image.depth: 1(int)
image.row: 28(int)
jobid: 1(int)
keep.test: no(string)
keep.train: no(string)
layer1: cnn(string)
layer2: flatten(string)
layer3: dense(string)
layer4: dense(string)
loss.type: categoricalcrossentropy(string)
nn.out: 5(int)
test_file: ../../dataset/mnist_test.dat(string)
train_file: ../../dataset/mnist_train.dat(string)
sucess read training dataset
success in read test dataset
finish reading
cnn
flatten
dense
dense
build model
=====
epoch = 1
epoch = 1  i= 0  loss = 1.64441
epoch = 1  i= 2  loss = 1.6223
epoch = 1 train loss = 1.64003 test loss  = 1.57718
train accuracy = 0.192708  test accuracy = 0.34375
cost time = 1.047 seconds.
```

训练后的结果保存在 model_1.h5 文件里。

9. 联系我们

如有使用问题或建议，请联系：李绍直 lishaozhiphys@gmail.com

注意：本软件为研究用途设计，建议用户在理解量子计算与神经网络基本原理的基础上使用。欢迎反馈与提供改进建议！

10. 版权归属

HQNN 由李绍直独立编写开发。使用 HQNN 需要经过开发者同意。